Tema 2:

Tipos Abstractos de Datos

GRADO EN INGENIERÍA DEL SOFTWARE

Salvador Sánchez y Cristian Rodríguez

Universidad
Rey Juan Carlos

Curso 2023-2024

Contenidos

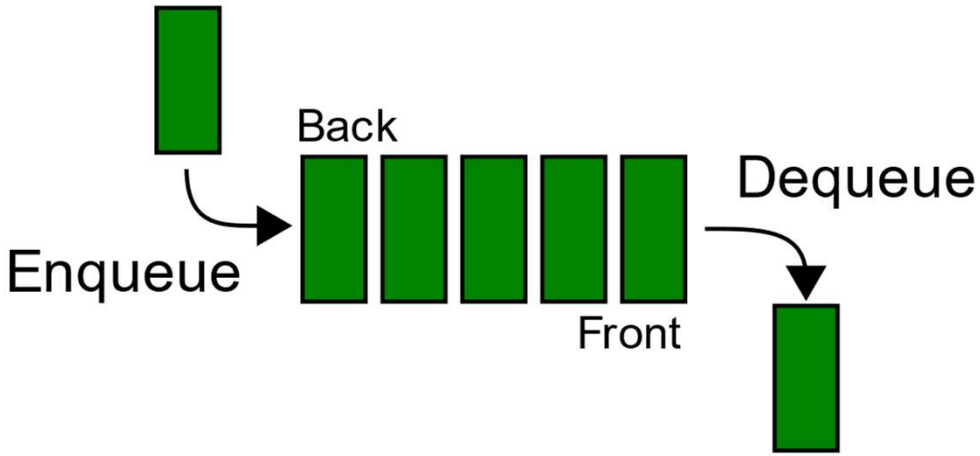
- Definición de TAD
- Especificación
- Implementación en Pascal
- Ejercicios propuestos

TAD

- **Un tipo de datos abstracto (TAD)** es una descripción de alto nivel de una colección de datos y de las operaciones que pueden realizarse con ella.
- Define la estructura de datos con un conjunto de reglas y comportamientos, sin especificar la implementación real.
- Esencial: independencia de la implementación

TAD: ejemplo

- Una cola de espera permite: entrar, salir, consultar si está vacía, consultar cuál es el último, consultar cuál es el primero.



- Internamente puede programarse de múltiples formas, pero externamente el comportamiento es siempre el mismo

TAD: tipos de operaciones

- Las operaciones de un TAD se pueden clasificar como:
  - **Creación**: Permiten inicializar y obtener nuevos objetos del tipo. En POO se denominan constructores.
  - **Modificación**: Permiten realizar cambios en el estado interno de un objeto del tipo
  - **Consulta**: Extraen información del TAD
  - **Propias del TAD**: Operaciones entre elementos del tipo no definidas para otros, p.ej. Determinante de una matriz, etc.

TAD: especificación

- Para definir un TAD se emplea una **especificación**
  - Debe ser independiente de cualquier implementación
  - Diferentes modos de especificación: lenguaje natural, especificación algebraica, lenguajes formales, etc.
- Consiste en separar el comportamiento esperado de su implementación concreta.
- Estamos definiendo cómo se comporta un nuevo tipo.

TAD: especificación

- Especificación en lenguaje natural:
  - **Nombre** del TAD.
  - **Valores** posibles.
  - **Operaciones** (algunas pueden tener restricciones que indicaremos mediante precondiciones).
  - **Uso** de otros tipos.

TAD: ejemplos de especificación

- **Nombre:** Semáforo.
- **Valores posibles:** *rojo*, *verde*.
- **Operaciones:**
  - abrir
  - cerrar
  - comprobar si está abierto
- **Uso de otros tipos:** char/enteros/boolean (según implementación).

TAD: ejemplos de especificación

- Implementación:
  - `abrir()`: procedimiento, pone valor interno a verde
  - `cerrar()`: procedimiento, pone valor interno a rojo
  - `esta_abierto()`: función, retorna true/false según valor interno

EJERCICIO

¿Cómo especificarías un semáforo con tres luces (verde, rojo y ámbar)?

TAD: ejemplos de especificación

- **Nombre:** Semáforo.
- **Valores posibles:** *rojo*, *verde, ámbar*.
- **Operaciones:**
  - abrir
  - cerrar
  - comprobar si está abierto
- **Uso de otros tipos:** char/entero (según implementación)

TAD: ejemplos de especificación

- Implementación:
  - `abrir()`: procedimiento, pone valor interno a verde
  - `cerrar()`: procedimiento, pone valor interno a ámbar durante 3 segundos, luego a rojo
  - `esta_abierto()`: función, retorna true/false según valor interno

TAD: ejemplos de especificación

- **Nombre:** Contador
- **Valores posibles:** enteros positivos
- **Operaciones:**
  - inicializar
  - incrementar
  - consultar su valor
  - decrementar (PRE: si no está a cero)
- **Uso de otros tipos:** enteros

TAD: ejemplos de especificación

- Implementación:
  - `inicializar()`: procedimiento, pone entero interno a cero
  - `incrementar()`: procedimiento, suma 1 al valor interno
  - `obtener_valor()`: función, retorna valor interno
  - `decrementar()`
    - PRE: si no está a cero
    - Implementación: procedimiento, decrementa en 1 el valor interno, si no está a cero

EJERCICIO

¿Cómo sería la especificación en lenguaje natural de un contador con límite y que puede incrementar en más de una unidad cada vez?

TAD: ejemplos de especificación

- **Nombre:** Contador_limitado
- **Valores posibles:** enteros positivos
- **Operaciones:**
  - Inicializar()
  - Incrementar(n) (*)
  - Consultar su valor
  - Decrementar (PRE: no está a cero)
- **Uso de otros tipos:** enteros

(*) Habrá que pensar qué hacer cuando se alcanza el límite

TAD: implementación

- Para la implementación de TADs en un lenguaje de programación se tienen en cuenta algunos detalles:
  - *Privacidad*: la representación interna y las operaciones tienen diferentes niveles de privacidad. En el más restrictivo el acceso está vetado al usuario externo, que no puede acceder directamente
  - *Genericidad*: definidos en función de un elemento interno que se especifica en la creación. Generalmente colecciones de elementos (cola de personas, cola de procesos…)
  - *Sobrecarga*: subprogramas con el mismo nombre pero diferencias en comportamiento y número de parámetros.
- Pascal no da soporte a estos conceptos, por lo que el usuario debe tener disciplina y utilizar el TAD como se indica en la interfaz, **NUNCA accediendo directamente a sus datos internos**

TAD: implementación en Pascal

- Para la implementación de tipos abstractos de datos en Pascal utilizaremos las unidades.
- Una unidad (UNIT) contiene una parte de interfaz y otra de implementación.
  - Interfaz pública (visible desde el programa externo)
  - Implementación privada (no visible desde fuera de la unit)
- El nombre de la unidad y el del archivo en que se encuentra **deben ser iguales**

TAD: implementación en Pascal

- Podemos compilar la unidad como si fuese un programa
- Para usar la unidad desde un programa simplemente añadimos la cláusula:

```
USES nombre_de_la_unidad
```

- `Uses` permite hacer disponible los elementos declarados en la parte pública de la unidad (interfaz)
- El fichero…
  - lo guardamos con el nombre dado a la unidad (extensión .PAS)
  - lo compilamos para generar un archivo .TPU (Turbo Pascal Unit)
  - la unidad está ahora accesible

TAD: implementación en Pascal

Interfaz (contrato) del TAD **Contador** implementado en Pascal:

```
UNIT uContador;

INTERFACE

  TYPE
     TContador = record
       valor: integer;
     end;

  PROCEDURE inc(VAR c: TContador);
  PROCEDURE dec(VAR c: TContador);
  PROCEDURE reset(VAR c: TContador);
  FUNCTION  get_valor(c: TContador): integer;
```

- Todo lo que aparece en la interfaz podrá ser utilizado por cualquier programa que haga uso de la unidad `uContador`

TAD: implementación en Pascal

Implementación del TAD **Contador** en Pascal:

```
IMPLEMENTATION

  PROCEDURE inc(VAR c: TContador);
  BEGIN
    c.valor := c.valor + 1
  END;

  PROCEDURE dec(VAR c: TContador);
  BEGIN
    if c.valor > 0 then c.valor := c.valor - 1
  END;

  PROCEDURE reset(VAR c: TContador);
  BEGIN
    c.valor := 0
  END;

  FUNCTION get_valor(c: TContador): integer;
  BEGIN
    get_valor := c.valor
  END;

END.
```

TAD: implementación en Pascal

Ejemplo de uso:

```
program prueba_contadores;
uses crt, uContador;
var
  x: TContador;
  i: integer;
begin
     reset(x);
     writeln(get_valor(x));
     for i:= 1 to 20 do inc(x);
     writeln(get_valor(x));
     repeat until keypressed
end.
```

EJERCICIO

Utilizando como base la implementación del contador:

a) Definir la interfaz de una unidad `uContadorLimitado` con las funciones adicionales (i) establecer un límite máximo tras el cual se pone a cero y (ii) poder incrementarlo en cualquier número de unidades.

b) Implementarlo

EJERCICIO: Solución

Interfaz del TAD **ContadorLimitado** en Pascal:

```
UNIT uContadorLimitado;

INTERFACE

  CONST LIMITE = 9999;

  TYPE
     TContador = record
       valor: integer;
     end;

  PROCEDURE inicializar(VAR c: TContador; valor_inicial:integer = 0);
  PROCEDURE inc(VAR c: TContador);
  PROCEDURE dec(VAR c: TContador);
  PROCEDURE reset(VAR c: TContador);
  PROCEDURE inc_valor(VAR c: TContador; n: integer);
  FUNCTION  get_valor(c: TContador): integer;
```

EJERCICIO: Solución

Implementación:

```
IMPLEMENTATION
  PROCEDURE inicializar(VAR c: TContador; valor_inicial: integer = 0);
  BEGIN
    c.valor := valor_inicial
  END;

  PROCEDURE inc(VAR c: TContador);
  BEGIN
    if c.valor < LIMITE then c.valor := c.valor + 1
    else c.valor := 0
  END;

  PROCEDURE dec(VAR c: TContador);
  BEGIN
    if c.valor > 0 then c.valor := c.valor - 1
  END;

  PROCEDURE reset(VAR c: TContador);
  BEGIN
    c.valor := 0
  END;

  PROCEDURE inc_valor(VAR c: TContador; n: integer);
  VAR
    i: integer;
  BEGIN
    for i:= 1 to n do inc(c)
  END;

  FUNCTION get_valor(c: TContador): integer;
  BEGIN
    get_valor := c.valor
  END;

END.
```

EJERCICIO: Solución

Uso del TAD **ContadorLimitado** en Pascal:

```
program prueba_contadores;
uses crt, uContadorLimitado;
var
  x: TContador;
  i: integer;
begin
     reset(x);
     writeln(get_valor(x));
     for i:= 1 to 20 do inc(x);
     writeln(get_valor(x));
     repeat until keypressed
end.
```

- Es IDÉNTICO al uso del contador no limitado ¿por qué?
  - *RESPUESTA: Ambos tienen mismo nombre de tipo e implementan misma interfaz*

Beneficios del uso de TADs

- El empleo de TADs promueve la separación de tareas:
  - El programa que usa el Contador solo tiene que conocer cómo se usa un Contador (su interfaz) no sus detalles de implementación
  - Nuestros programas pueden centrarse en su código de negocio e ignorar los detalles de las estructuras auxiliares que manejan (contadores, pilas, diccionarios, árboles, etc.)
- El mantenimiento del código se vuelve más simple: el código cliente no debería cambiar ante cambios en la implementación de un TAD.
- El uso de TADs favorece la reutilización de código ya probado y promueve código más legible
- Al extremo, la construcción de programas se limitará a interconectar módulos existentes y escribir un poco de código de negocio

Ejercicios propuestos

Ejercicio propuesto 1

- Especificar e implementar un TAD **`punto3D`**:
  - Crear el punto con valores dados (por defecto el origen)
  - Modificar cada coordenada (`setx`, `sety`, `setz`)
  - Obtener el valor de una coordenada (`getx`, `gety`, `getz`)
  - Longitud del vector que va del origen al punto (módulo)
  - Distancia a otro punto

Ejercicio propuesto 2

- Especificar e implementar un TAD **`cString`** que proporcione en Pascal las siguientes funcionalidades:
  - Crear el `cString` con un cierto tamaño
  - Darle valor
  - Obtener su valor
  - Obtener su longitud (`strlen`)
  - Obtener su capacidad máxima de almacenamiento (tamaño)
  - Implementar las funciones más comunes de manejo cadenas en el lenguaje C: `strcat`, `strcpy`, `strcmp`, `strlwr` y `strupr`

Ejercicio propuesto 3

- Especificar e implementar un TAD **`complejo`** que de soporte al concepto matemático de número complejo:
  - Crear el `complejo` con valores dados (por defecto i=0, r=0)
  - Darle valor a su parte real
  - Darle valor a su parte imaginaria
  - Obtener parte real
  - Obtener parte imaginaria
  - Implementar las funciones más comunes: `suma, resta, multiplicación, división, potencia`

Ejercicio propuesto 4

- Especificar e implementar un TAD `TarjetaCredito` que de soporte a las siguientes operaciones:
  - Inicializar los datos de una tarjeta para comenzar a operar
  - Consultar el nombre del titular
  - Consultar el nombre del banco a cuya cuenta carga los pagos
  - Obtener el número de la tarjeta
  - Consultar el límite de crédito
  - Consultar el saldo actual de la tarjeta
  - Cargar un pago